

PitStop System Architecture and Operational Flows

Document type: Production architecture & flow specification

Audience: Engineers, product stakeholders, operations, and auditors

Stack: Expo Router (React Native / web) · Supabase Auth · PostgreSQL · RLS · Realtime · Storage · Edge Functions

Canonical schema sources: `supabase/schema.sql` , `supabase/complete_system_schema.sql` , incremental `apply-pitstop-2.0-step*.sql`

Last aligned to codebase: 20 August 2026

Production retail update: store owner 5-tab shell, completed-only revenue, order restock on cancel, customer `/((tabs)/orders` tab, tax invoice receipt, multi-image SKUs, and cart stock caps. See the delta section below plus 2.2–2.7, 3.1, 3.3–3.5, Appendix A, and Appendix C.

Table of contents

1. [August 2026 production retail delta](#)
 2. [Executive overview and platform roles](#)
 3. [Owner / merchant ecosystem and dashboards](#)
 4. [Customer user journey and checkout flow](#)
 5. [Super admin governance panel](#)
 6. [Database schema and Supabase topology](#)
 7. [Realtime event pipelines and edge integrations](#)
-

August 2026 production retail delta

This section is the current source of truth for retail work shipped after the 18 August 2026 snapshot. Service shop shells (`wash` / `maintenance` / `winch`) stay on the 4-tab layout unless a later section says otherwise.

Store owner shell (5 tabs)

- Tab ids come from `lib/owner/dashboardConfig.ts` `getOwnerNavTabs(shopType)` .
- **Store** (`parts` , `accessories`): Overview, Orders, Inventory (AR `المخزن`), Profile, Settings.
- **Service**: Overview, Bookings, Services, combined Profile & Settings. Header cog still opens `/shop/merchant-settings` .
- Store header cog switches to the in-shell Settings tab (`components/store/owner/StoreOwnerSettings.tsx`).
- Profile is customer-facing shop info + GPS pin: `components/store/owner/StoreOwnerProfileSections.tsx` .
- Operating status Open / Closed / Maintenance is persisted as `shop_extras.storeOperatingStatus` via `setStoreOperatingStatus` in `lib/booking/shopExtrasStorage.ts` . Overview shows a read-only chip that navigates to Settings.
- Metric cards: pending → Orders filter, revenue → `/shop/store-reports` , low stock → Inventory filter (`lib/store/ownerFilters.ts`).

Order lifecycle, revenue, restock

- DB enum remains `preparing` ; UI copy is Processing / قيد التجهيز .
- Owner RPC `update_store_order_status` (`supabase/store_order_lifecycle_refactor.sql`): `pending` → `preparing` | `ready` | `cancelled`; `preparing` → `ready` | `cancelled`; `ready` → `completed` | `cancelled`; `completed` → `cancelled` (void).
- Any transition to `cancelled` restocks line items atomically (row lock + increment `stock_quantity`).
- Revenue in `get_store_owner_stats` and `get_store_sales_report` counts `status = completed` only.
- Customer cancel (pending only): RPC `cancel_own_store_order` (`supabase/customer_cancel_store_order.sql`) — same restock path. Owner RPC is still used for merchant status changes.

Customer My Orders + tax invoice

- Dedicated tab `app/(tabs)/orders.tsx` , registered in `app/(tabs)/_layout.tsx` (icon `bag-handle-outline`).
- Customer tab map: Home, Bookings, **Orders**, Favorites, Driver Network, Account. Bookings is service bookings only.
- List: `components/customer/CustomerStoreOrdersList.tsx` via `listStoreOrdersForUser(userId)` in `lib/store/productRepository.ts` . Cards: `CustomerStoreOrderCard.tsx` .
- Actions: cancel pending, Call / WhatsApp (`lib/linking/contact.ts` `openWhatsAppTo`), Buy again via `StoreCartContext.addProduct` .
- Tax invoice modal: `components/customer/StoreOrderInvoiceModal.tsx` . Print/Save: `lib/store/storeOrderInvoice.ts` + `expo-print` . Title "Tax Invoice / فاتورة شراء" . Pickup delivery line is Free / مجاناً .

Catalog, cart, checkout

- SKU media: `products.image_url` (cover) + `products.image_urls` `text[]` max 5. Customer slider: `StoreProductImageSlider` . Helpers: `lib/store/productImages.ts` . Instant local previews in `AddProductModal` ; upload on save.
- Cart stock caps: `lib/store/stockLimits.ts` + `StoreQuantityStepper` . Checkout overstock banner; RPC still raises insufficient-stock on race.
- `place_store_order(..., p_shop_id)` scopes cart lines to one merchant so mixed carts can be fulfilled one shop at a time.

Canonical SQL / client files for this delta

Concern	File
Lifecycle + restock + completed-only revenue	<code>supabase/store_order_lifecycle_refactor.sql</code>
Multi-image SKUs, per-shop checkout, sales report	<code>supabase/store_enhancements_production.sql</code>
Customer cancel	<code>supabase/customer_cancel_store_order.sql</code>
Product-images Storage RLS	<code>supabase/fix_storage_rls_product_images.sql</code> (if present in repo)
Owner nav	<code>lib/owner/dashboardConfig.ts</code> , <code>components/owner/OwnerDashboardNav.tsx</code>
Store owner settings / profile	<code>StoreOwnerSettings.tsx</code> , <code>StoreOwnerProfileSections.tsx</code>

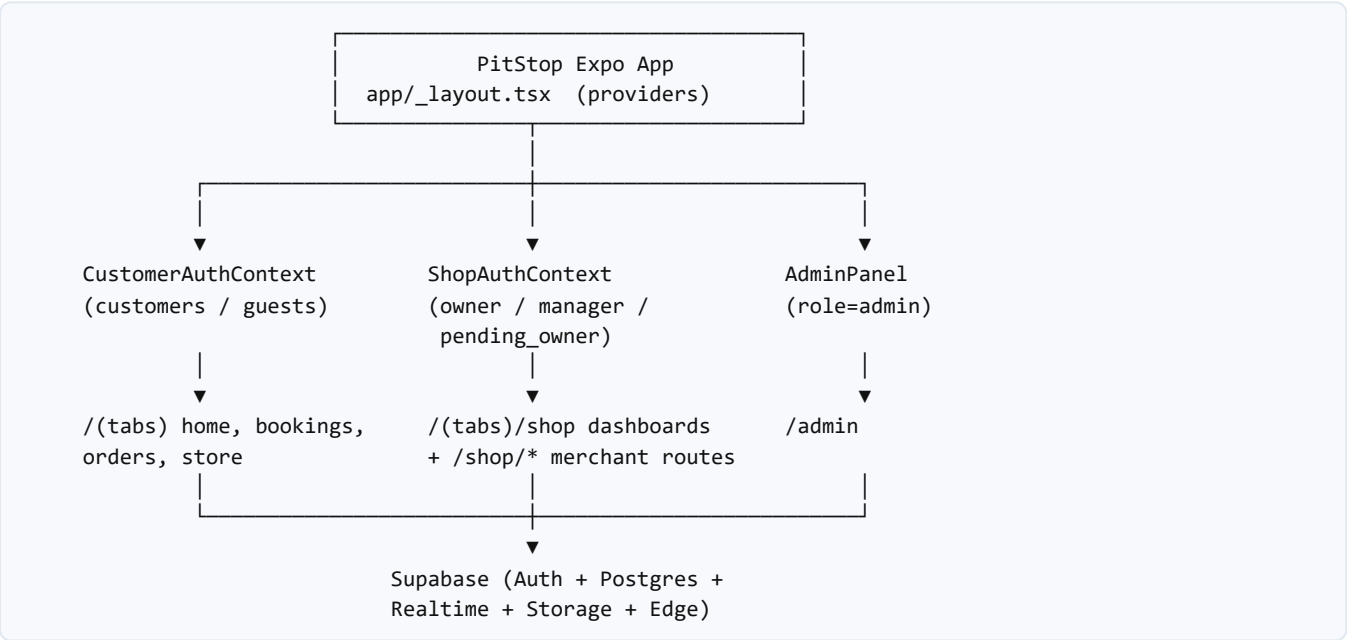
1. Executive overview & platform roles

1.1 Platform purpose

PitStop is a multi-tenant marketplace for automotive services and retail in Egypt. One Expo app serves three distinct populations from a shared Supabase project:

Population	Primary job	Typical entry
Customers	Discover shops, book services, buy parts/accessories, track orders	/welcome → customer login or guest → /(tabs)
Merchants / owners	Operate a shop: queue, catalog, inventory, fulfillment, settings	/welcome?focus=owner → /(tabs)/shop
Super admins	Approve merchants, toggle premium, moderate content, remove accounts	Owner login with users.role = admin → /admin

The domain is **shop-centric**: every merchant brand is a row in public.shops . Physical locations are public.shop_branches . Customers never own shops; they own bookings, carts, orders, vehicles, and community posts.



1.2 Multi-tenant domain model

Tenancy is **shop_id scoped**, not organization-id scoped.

```
auth.users (Supabase Auth identity)
  | 1:1 (handle_new_auth_user trigger)
  ▼
public.users  role, shop_id, branch_id, is_active
├── customer      → no shop_id
├── pending_owner → shop_id set, shop.is_active = false
├── owner         → shop_id required
├── branch_manager → shop_id + branch_id required
└── admin        → no shop_id

public.shops (tenant root)
├── shop_branches (physical bays / locations)
│   ├── branch_services
│   └── branch_employees (no login)
├── products      (parts / accessories merchants)
└── store_orders
```

- bookings
- shop_extras (JSONB profile, hours, gallery)
- offers / coupons / campaigns
- shop_push_tokens

Shop types (`public.shop_type`):

Enum value	Merchant category	Customer surface	Owner surface
wash	Car wash	Area list → shop profile → /book/[shopId]	WashOwnerPanel
maintenance	Service center	Same booking path	app/(tabs)/shop.tsx service dashboard
winch	Roadside / winch	Same booking path	Service dashboard + winch phone toggle
parts	Spare-parts retailer	/store?category=spare_parts	StoreOwnerDashboard
accessories	Accessories retailer	/store?category=accessories	StoreOwnerDashboard

`shops.category` is a generated compatibility alias: `wash` → `car_wash`, `parts` → `spare_parts`, otherwise `type::text` (`complete_system_schema.sql` trigger `shops_sync_category`).

1.3 Role definitions

App identity lives in `public.users`. TypeScript mirror: `lib/supabase/database.types.ts` → `DbUserRole`.

Role	Login context	Session resolver	Can do	Cannot do
customer	CustomerAuthContext	Confirmed <code>auth.users</code> email that is not a shop staff email	Browse, book, cart, checkout, reviews, community	Open merchant dashboard
Guest	<code>continueAsGuest()</code>	Local flag only, no <code>auth.uid()</code>	Browse catalog / map	Checkout, booking persist, cart persist
pending_owner	<code>ShopAuthContext.login</code>	<code>resolveShopSession</code>	Wait after register	Operate dashboard (<code>pending_approval</code>)
owner	<code>ShopAuthContext</code>	<code>users.role=owner</code> + <code>shop_id</code>	Full shop + all branches (premium for extra branches)	Admin approval RPCs
branch_manager	<code>ShopAuthContext</code>	<code>users.role=branch_manager</code> + <code>branch_id</code>	Queue, walk-in POS, branch-scoped bookings	Other branches, premium toggles, delete shop
admin	Same merchant login form	<code>users.role=admin</code>	Approval, premium, moderation, merchant delete	Impersonate customer checkout

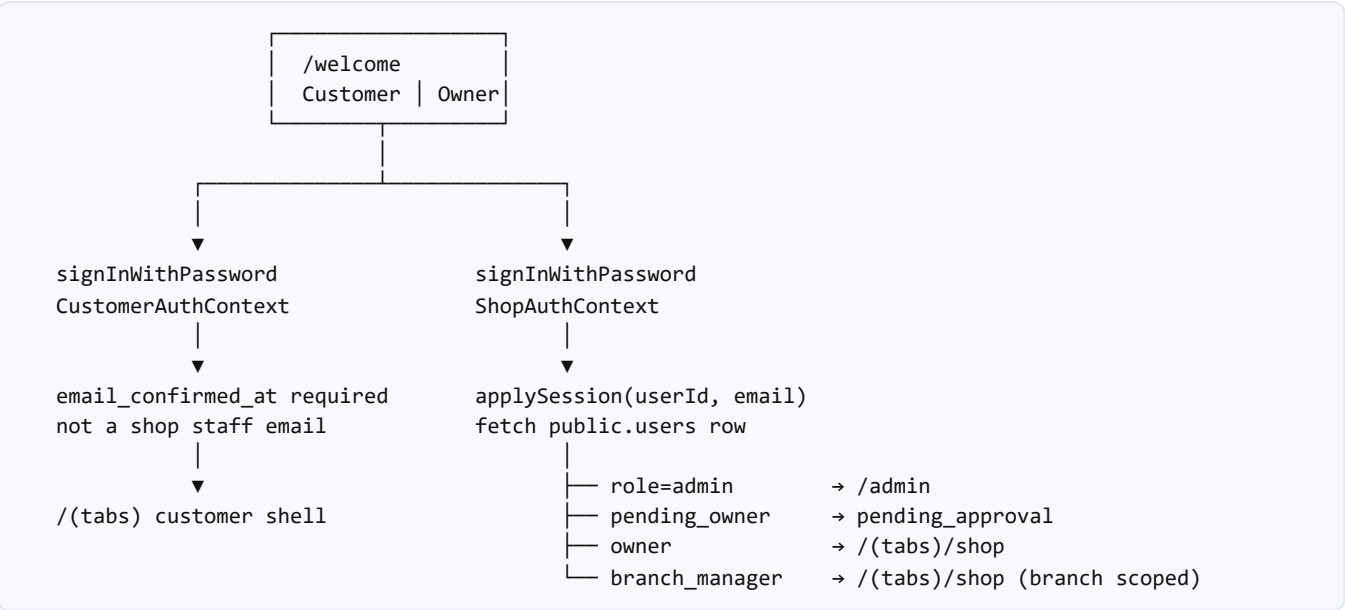
Staff vs customer isolation

- `CustomerAuthContext.resolveCustomerFromUser` rejects emails that `isShopStaffEmailRemote` reports as shop staff.
- `ShopAuthContext.login` returns `shop_not_found` if the session is not owner/manager/admin/pending_owner.

- Admins are redirected: `app/(tabs)/shop.tsx` → `router.replace('/admin')` .

1.4 Authentication models

Both populations use **Supabase Auth email/password**. Sessions are isolated per browser tab via `lib/storage/webTabAuthStorage.ts` and `lib/supabase/webTabAuthIsolation.ts` .



Customer register: `CustomerAuthContext.register` → `auth.signUp` → `handle_new_auth_user` inserts `public.users` with `role='customer'` .

Merchant register: `lib/shop/ownerRegistrationRepository.ts` → `auth.signUp` (metadata `account_type=shop_owner`) → RPC `register_shop_owner(...)` creates an **inactive** shop and sets `users.role='pending_owner'` . Login then returns `pending_approval` until a super admin accepts.

Password reset: `/reset-password` (customers). Merchant password change: `/shop/merchant-password` .

Account deletion: customers call `deleteCustomerAccountRemote` (`lib/account/customerAccountRepository.ts`); merchants are removed only by admin (`deleteMerchant`).

1.5 Permission boundaries (enforced in Postgres)

Helper functions (security definer, `search_path = public`):

Function	True when
<code>is_platform_admin()</code>	<code>users.role = admin</code> and <code>is_active</code>
<code>is_shop_owner(p_shop_id)</code>	Active owner row for that shop or JWT email matches <code>shops.owner_email</code>
<code>is_branch_manager(p_branch_id)</code>	Active <code>branch_manager</code> with that <code>branch_id</code>
<code>can_manage_shop(p_shop_id)</code>	Owner or any branch manager of that shop
<code>can_manage_branch(p_branch_id)</code>	Branch manager of that branch or shop owner

RLS then uses those helpers. Application code never trusts the client for stock decrement or order insert: checkout goes through `place_store_order` .

1.6 Provider tree (`app/_layout.tsx`)

```
ThemeProvider
  I18nProvider
    AppDialogProvider / CustomConfirmProvider
      ShopCatalogProvider
        CustomerAuthProvider
          StoreCartProvider
```

```
ShopAuthProvider
AppBootstrap
RootStack (expo-router Stack)
```

Initial route: `welcome` (`unstable_settings.initialRouteName`).

2. Owner / merchant ecosystem & dashboards

Owner chrome is role- and type-routed from a single tab: `app/(tabs)/shop.tsx`.

```
/(tabs)/shop
├── not logged in → email/password merchant login form
├── isAdmin → redirect /admin
├── shop.type = wash → components/owner/wash/WashOwnerPanel.tsx
├── parts|accessories → Store owner shell (5 tabs via getOwnerNavTabs)
└── maintenance|winch → Service owner shell (4 tabs)
```

Shared bottom navigation: `components/owner/OwnerDashboardNav.tsx`. Tab ids come from `lib/owner/dashboardConfig.ts` → `getOwnerNavTabs(shopType)`. **Store merchants have five tabs** (Profile and Settings are split). Service merchants keep four tabs with combined Profile & Settings.

Tab id	Store merchants (parts , accessories)	Service merchants (wash , maintenance , winch)
dashboard	Overview / نظرة عامة — metrics + read-only status	Overview + metrics
management	Orders / الطلبات — <code>StoreOrdersPanel</code>	Bookings / queue
operations	Inventory / المخزن — <code>StoreInventoryManager</code>	Services catalog
profile	Profile / الملف التعريفي — customer-facing shop info, cover, avatar, gallery, GPS pin	Combined Profile & Settings
settings	Settings / الإعدادات — Open/Closed/Maintenance, hours, password, locale, legal, logout	<i>n/a — cog still opens /shop/merchant-settings</i>

2.1 Merchant categories

A. Service providers — car wash

Primary component: `components/owner/wash/WashOwnerPanel.tsx`

Satellite screens:

Route	File	Purpose	
<code>`/shop/wash-owner-hub?tab=orders`</code>	<code>reviews`</code>	<code>app/shop/wash-owner-hub.tsx</code>	Notification inbox + pending queue + reviews
<code>/shop/wash-reports</code>	<code>app/shop/wash-reports.tsx</code>	Revenue / analytics reports	
<code>/shop/wash-history</code>	<code>app/shop/wash-history.tsx</code>	Historical bookings	
<code>/shop/merchant-hours</code>	<code>app/shop/merchant-hours.tsx</code>	Hours editor (settings deep-link)	
<code>/shop/merchant-services</code>	<code>app/shop/merchant-services.tsx</code>	Services editor	

Route	File	Purpose
/shop/merchant-staff	app/shop/merchant-staff.tsx	Staff / manager

Operating unit: a **branch** (`shop_branches`), not a literal “bay” table. Each branch is an active service location with:

- `shop_status`: `open` | `closed` | `busy` | `vacation`
- `weekly_hours` JSONB → slot engine in `lib/booking/shopSchedule.ts`
- `service_duration_minutes` (default 60 in schema, often 30 in wash UI)
- `vacation_mode` JSONB
- GPS (`latitude` , `longitude`)
- Cover + gallery (`profile_image_url` , `image_urls`)

Premium (`shops.is_premium`) unlocks extra branches, coupons, campaigns, and several CMS sections via `components/owner/PremiumFeatureGate.tsx` . Free owners keep one branch (`premium_branch_free_hint`).

B. Service providers — maintenance / winch

Rendered inside `app/(tabs)/shop.tsx` (not `WashOwnerPanel`). Same four tabs. Extra: winch enable/phone on maintenance shops (`shop_manage_winch_*`). Schedule is shop-level `shop_extras` (`workOpenTime` , `workCloseTime` , `serviceDurationMinutes` , `weeklyHours`).

C. Retail / product stores — spare parts & accessories

Detected by `isStoreShopType()` in `lib/booking/storeCatalog.ts` . Catalog category mapping:

- `parts` → `spare_parts`
- `accessories` → `accessories`

Products are **merchant-owned** (`products.shop_id` NOT NULL after `complete_system_schema.sql`). Checkout is single-merchant: `place_store_order` rejects mixed-shop carts.

2.2 Layout architecture (all merchant types)

OwnerProfileHeader (cover, avatar, notifications, cog)
Tab body (ScrollView) dashboard management operations profile settings (store: 5 tabs; service: 4 tabs, no standalone settings)
OwnerDashboardNav (absolute bottom bar)

For **store** owners, the header cog switches to the in-shell Settings tab. Account password / terms / privacy also remain at `/shop/merchant-settings` . Service owners still open that route from the cog. Customer tab bar is **hidden** while a shop session is active (`app/(tabs)/_layout.tsx`).

2.3 Overview & key metrics

Store merchants

Component: `components/store/owner/StoreOwnerDashboard.tsx`

Data: RPC `get_store_owner_stats(p_shop_id)` with table fallback in `lib/store/storeStatsRepository.ts` .

Metric id	Label key	Source	Notes
revenue	store_owner_total_revenue	sum(store_orders.total_price) where status = completed only	Pending / preparing / ready are operational, not revenue. Sales report RPC matches this rule.
orders	store_owner_total_orders	count(*) of shop orders	All statuses
pending	store_owner_pending_orders	status = pending	Warning tone
stock	store_owner_low_stock	active products with stock_quantity < 5	Danger if > 0

Overview shows a **read-only** store status chip (Open / Closed / Maintenance) that navigates to Settings. Status is persisted on `shop_extras.storeOperatingStatus` via `setStoreOperatingStatus`. Metric cards are tappable: pending → Orders filter, revenue → `/shop/store-reports`, low stock → Inventory filter.

Realtime: channel `store-dashboard:{shopId}` listens to `store_orders` and `products` filtered by `shop_id`.

Wash merchants

Analytics: `lib/booking/wash/washAnalytics.ts` → `WashAnalyticsSnapshot`

Field	Meaning
todayBookings	Bookings scheduled today
pendingRequests	status = pending in staff scope
monthlyRevenue / weeklyRevenue	Done bookings that count as revenue (countsAsRevenueBooking)
averageRating	shop_reviews
mostBookedService / peakHourLabel / bookingTrend	Queue analytics

Workspace vs history chips on the dashboard tab: `owner_panel_tab_workspace` / `owner_panel_tab_history` (`OwnerHistoryPanel`).

Maintenance / winch

Computed in `app/(tabs)/shop.tsx` → `serviceOwnerMetrics`:

Metric	Rule
Revenue	Sum <code>servicePriceEgp</code> where <code>status = done</code>
Today bookings	<code>scheduledAt</code> date = today
Pending requests	status = pending
Active services	<code>shop_extras.services</code> where <code>active</code>

2.4 Inventory & catalog management

Retail catalog (parts / accessories)

Component: `components/store/owner/StoreInventoryManager.tsx`

Repository: `lib/store/productRepository.ts`

Capability	Implementation
List including hidden	<code>listStoreProductsByShop(shopId, { includeInactive: true })</code>
Create	<code>createStoreProduct</code> → <code>products</code> insert

Capability	Implementation
Inline edit	price, sale price, stock
Atomic live stock	+/- stepper then <code>updateStoreProductFields</code> ; DB check <code>stock_quantity >= 0</code>
Base vs sale price	<code>price</code> , <code>sale_price</code> with CHECK <code>sale_price IS NULL OR (sale_price >= 0 AND sale_price <= price)</code>
Visibility	<code>is_active</code> toggle
Category tagging	Shop type → <code>store_product_category</code> ; sub-category from <code>lib/store/constants.ts</code> <code>STORE_SUB_CATEGORIES</code>
Compatibility	<code>product_compatibility</code> rows (<code>universal</code> / <code>brand_specific</code> / <code>model_specific</code>)
Image upload	<code>uploadImageToBucket</code> → bucket <code>product-images</code> . Instant local previews in <code>AddProductModal</code> ; upload on save. Web uses File/Blob + <code>window.fetch</code> .
Product images	<code>products.image_url1</code> (cover) plus <code>products.image_urls</code> (up to 5). Customer UI: <code>StoreProductImageSlider</code> . Helpers: <code>lib/store/productImages.ts</code> .
Delete	<code>deleteStoreProduct(productId, shopId)</code>
Low stock UI	Visual when stock < 5
Realtime	channel <code>store-inventory:{shopId}</code> on <code>products</code>

Stock caps (customer cart): `lib/store/stockLimits.ts` clamps quantity to live `stock_quantity` .

`StoreQuantityStepper` cannot exceed stock. Checkout shows an overstock banner; `place_store_order` still raises insufficient-stock if the race is lost.

Sub-categories (canonical ids):

<code>spare_parts</code>	<code>accessories</code>
<code>engine_oil</code> , <code>air_filters</code> , <code>brakes</code> , <code>fluids</code> , <code>batteries</code>	<code>car_care</code> , <code>electronics</code> , <code>interior</code> , <code>exterior</code>

Effective customer price (`lib/store/pricing.ts`): product `sale_price` wins over shop-level `storeGlobalDiscountPercent` ON `shop_extras` .

Admin catalog override: `/admin/store-products` (`app/admin/store-products.tsx`) + `components/store/admin/StoreProductForm.tsx` (uploads historically used bucket `shop-images`).

Service catalog (wash / maintenance)

Surface	Persistence
Wash services CRUD	<code>branch_services</code> via <code>lib/booking/wash/washBranchStorage.ts</code> <code>saveWashBranchServices</code>
Maintenance services	<code>shop_extras.services</code> JSON via <code>lib/booking/shopExtrasStorage.ts</code>
Categories (wash)	<code>exterior_wash</code> , <code>interior_cleaning</code> , <code>full_package</code> , <code>detailing</code> , <code>engine_cleaning</code> , <code>custom</code>
Visibility	<code>visible</code> / <code>active</code> flags; hidden items stay in owner list
Duration	<code>duration_minutes</code> drives slot occupancy

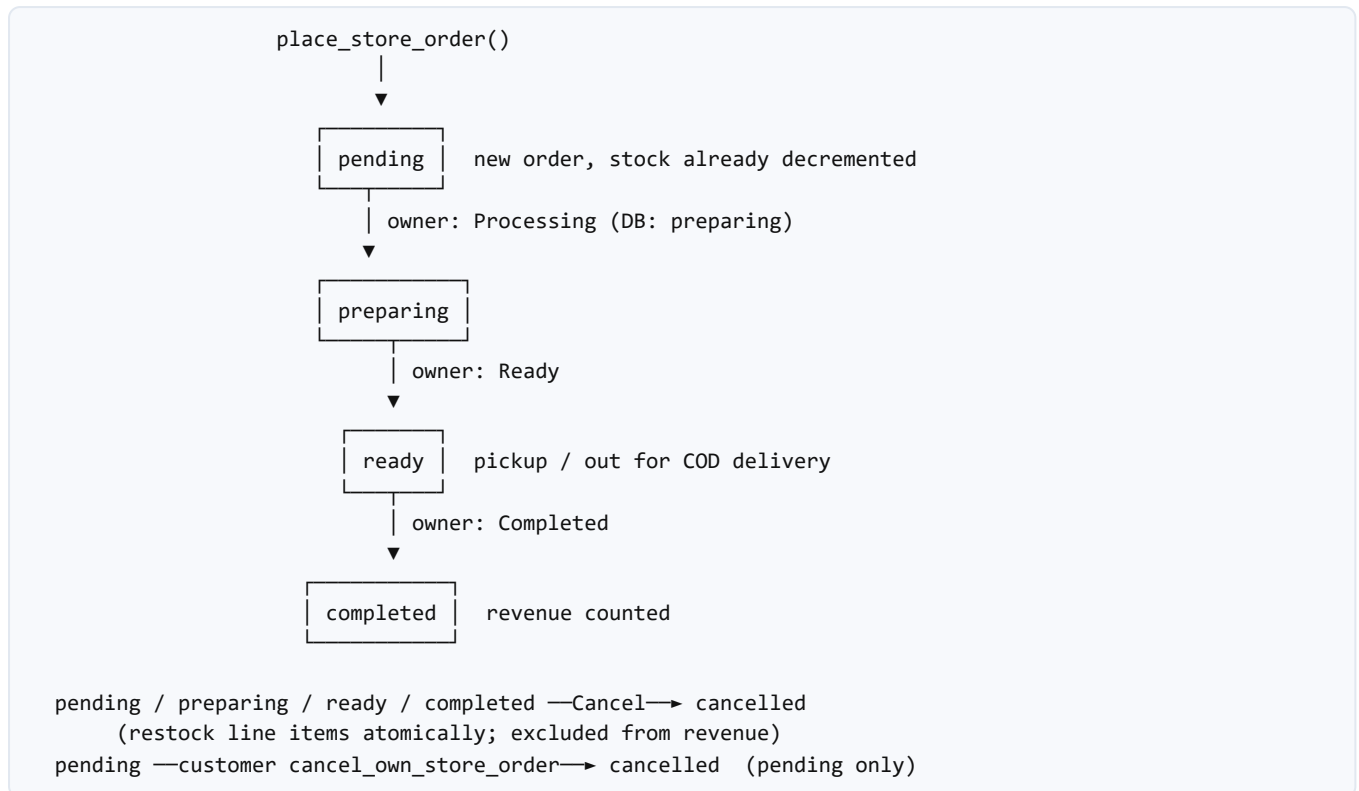
2.5 Order & fulfillment management (retail)

Component: `components/store/owner/StoreOrdersPanel.tsx`

Repository: `lib/store/orderRepository.ts`

Status RPC: `update_store_order_status(p_order_id, p_new_status)`

Order lifecycle



UI actions in `StoreOrdersPanel` (order detail modal). DB enum stays `preparing`; UI copy is `Processing` / قيد التجهيز.

Current status	Allowed owner actions
pending	preparing, ready, cancelled
preparing	ready, cancelled
ready	completed, cancelled
completed	cancelled (void + restock)
cancelled	none

Order payload shown to owner

- Short id `#` + first 8 UUID chars
- Customer name / phone (`customer_name` , `customer_phone`)
- Delivery address + notes (COD)
- Line items (`store_order_items` : name, qty, unit price, line total)
- Subtotal, delivery fee, grand total
- Fulfillment: `cod` (cash on delivery, fee `STORE_COD_DELIVERY_FEE_EGP` = 50) or `pickup` (fee 0)
- Enum also allows `card`; customer checkout UI currently exposes **COD and pickup only** (`StoreCartCheckoutModal`)

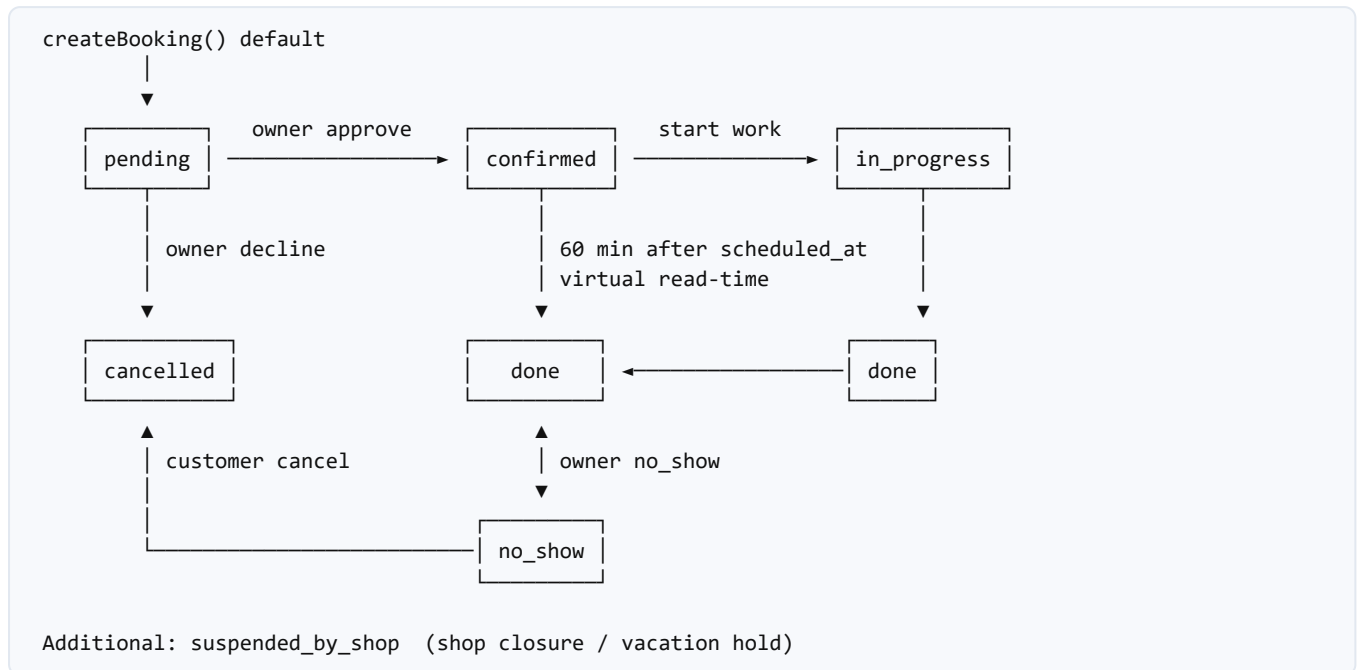
Realtime: channel `store-orders:{shopId}` on `store_orders` .

There is **no** `delivery_zones` table. Delivery is: COD + free-text address + flat 50 EGP fee, or in-store pickup. Geographic discovery uses shop/branch lat/lng and PostGIS proximity (`apply-pitstop-2.0-step15a-postgis-proximity.sql`), not zone polygons.

2.6 Service & queue management

Booking status machine

Persisted enum `public.booking_status` . TypeScript: `lib/booking/types.ts` .



Virtual lifecycle (`lib/booking/storage.ts`): `confirmed` or `in_progress` bookings whose `scheduled_at` is older than **60 minutes** (`AUTO_DONE_AFTER_MS`) are **read as done** with `lifecycleAutoCompleted=true` . Revenue reports **exclude** those virtual completions (`countsAsRevenueBooking`).

Booking types: `app` (customer) vs `walk_in` (POS). Walk-in skips customer confirmation email (`handleBookingConfirmed`).

Time slots

`lib/booking/shopSchedule.ts` :

- Builds slots from weekly hours + `serviceDurationMinutes`
- Occupancy from existing bookings on that shop/date
- Slot availability: `available` | `almost_full` | `booked`
- Customer UI: `BookingDatePicker` + slot list on `app/book/[shopId].tsx`

Cairo timezone is used for generated columns `booking_date` and `time_slot` (`Africa/Cairo` in `sync_booking_compatibility_columns`).

Active locations (bays)

Concept	Storage	UI
Active branch	<code>shop_branches.is_active</code> , <code>WashBranchState.activeBranchId</code>	Horizontal branch tabs in wash dashboard
Operating status	<code>shop_branches.shop_status</code>	Open / closed / busy / vacation
Staff scope	<code>users.branch_id</code>	Branch managers only see their branch queue (<code>filterPendingQueueBookingsForStaff</code>)
Walk-in POS	<code>WalkInBookingModal</code>	Creates <code>booking_type=walk_in</code>

Realtime queue: `owner-bookings:{shopId}` in `WashOwnerPanel` ; shared subscriber `subscribeMerchantBookingRealtime` in `lib/notifications/notificationService.ts` (INSERT + UPDATE on

bookings).

Side effects on status change (`updateBookingStatus`):

New status	Side effects
confirmed	<code>handleBookingConfirmed</code> → customer notification + <code>sendBookingConfirmationEmail</code>
cancelled	Merchant cancelled-booking push
done	Wash loyalty stamps + merchant loyalty points earn

2.7 Store / shop settings

Concern	Wash / branch	Maintenance	Retail store
Profile (EN/AR name, phone, email, address, more info)	Branch fields via <code>updateActiveWashBranch</code>	<code>setShopProfileInfo</code> → <code>shop_extras</code>	Same <code>shop_extras</code> profile card
Working hours	Per-day <code>weekly_hours</code> on branch	<code>setShopSchedule</code> / <code>setShopWeeklyHours</code>	Settings tab hours + <code>storeOperatingStatus</code> (Open/Closed/Maintenance). Checkout still requires <code>shops.is_active</code> .
Gallery	<code>image_urls</code> on branch	<code>shop_extras.imageUrls</code>	Same album on profile tab
Cover / avatar	<code>profile_image_url</code>	<code>profileImageUrl</code>	Same
Map pin	GPS capture + <code>OsmLocationPicker</code>	Shop lat/lng on <code>shops</code>	Profile tab GPS pin → shop lat/lng for store map
Delivery	N/A	N/A	COD address at checkout; pickup at shop
Staff	Branch manager invite + <code>branch_employees</code>	<code>/shop/merchant-staff</code>	Owner only
Legal	<code>/shop/merchant-terms</code> , <code>/shop/merchant-privacy</code>	same	Settings tab rows + <code>/shop/merchant-terms</code> , <code>/shop/merchant-privacy</code>
Operating status	Branch <code>shop_status</code>	Shop extras / hours	<code>shop_extras.storeOperatingStatus</code> via Settings tab

Retail Settings UI: `components/store/owner/StoreOwnerSettings.tsx` (status, hours, password, language, legal, logout). Retail Profile UI: `components/store/owner/StoreOwnerProfileSections.tsx` .

Campaigns panel (`components/merchant/MerchantCampaignsPanel.tsx`) is embedded on profile/operations depending on type.

3. Customer user journey & checkout flow

3.1 Customer shell routes

Route	File	Role
<code>/welcome</code>	<code>app/welcome.tsx</code>	Customer / merchant login & register
<code>/(tabs)</code>	<code>app/(tabs)/index.tsx</code>	Home: service tiles, store tiles, nearby, offers
<code>/(tabs)/bookings</code>	<code>app/(tabs)/bookings.tsx</code>	My Bookings — service bookings only
<code>/(tabs)/orders</code>	<code>app/(tabs)/orders.tsx</code>	My Orders — retail store orders

Route	File	Role
<code>/ (tabs)/favorites</code>	<code>app/(tabs)/favorites.tsx</code>	Saved shops
<code>/ (tabs)/assistant</code>	<code>app/(tabs)/assistant.tsx</code>	Driver network / community feed
<code>/ (tabs)/settings</code>	<code>app/(tabs)/settings.tsx</code>	Account (customers)
<code>/service/[type]</code>	<code>app/service/[type]/index.tsx</code>	Area picker
<code>/service/[type]/[areaId]</code>	<code>app/service/[type]/[areaId].tsx</code>	Shop list
<code>/nearby/[type]</code>	<code>app/nearby/[type].tsx</code>	Map / proximity
<code>/shop-profile/[shopId]</code>	<code>app/shop-profile/[shopId].tsx</code>	Public shop profile
<code>/book/[shopId]</code>	<code>app/book/[shopId].tsx</code>	Service checkout
<code>/booking/[id]</code>	<code>app/booking/[id].tsx</code>	Booking detail
<code>/store</code>	<code>app/store.tsx</code>	Product catalog + cart
<code>/parts-shop/[shopId]</code>	<code>app/parts-shop/[shopId].tsx</code>	Single-merchant storefront
<code>/settings/vehicles</code>	<code>app/settings/vehicles.tsx</code>	Garage
<code>/points-marketplace</code>	<code>app/points-marketplace.tsx</code>	Loyalty redemption catalog
<code>/auth-required</code>	<code>app/auth-required.tsx</code>	Guest gate
<code>/driver-network/[postId]</code>	<code>app/driver-network/[postId].tsx</code>	Community post

Home tiles (`app/(tabs)/index.tsx`):

- Services: `/service/maintenance` , `/service/wash`
- Store: `/store?category=spare_parts` OR `accessories`

3.2 End-to-end: service booking

```

Home → /service/{wash|maintenance|winch}
      → pick area
      → /service/{type}/{areaId} (list) or /nearby/{type} (map)
      → /shop-profile/{shopId}
      → /book/{shopId}
          |
          |─ pick vehicle (ActiveVehiclePicker / user_vehicles)
          |─ multi-select services (ServiceMultiPicker)
          |─ pick date + generated time slot
          |─ apply offer / coupon / loyalty points
          └─ createBooking() → public.bookings INSERT status=pending
              |
              |─ notifyWashOwnerBooking / Expo push (shop_push_tokens)
              |─ realtime INSERT → merchant OrderNotifier
              └─ owner confirms → email + customer notification

```

Guest: `/book/[shopId]` redirects through `/auth-required` with return path `buildBookReturnTo` .

Pricing at reservation (`lib/booking/storage.ts` `createBooking`):

1. Sum selected services (or shop default price).
2. Validate offer (`validateOfferForBooking`) — percentage, flat, BOGO, buy-X-get-Y.
3. Optional coupon (`validateCouponForCheckout` + `registerCouponUsageRemote` ; rolls back booking if usage log fails).
4. Optional loyalty (`validatePointsRedemptionRemote` / `deductMerchantLoyaltyPointsRemote`).
5. Platform fee = 12% of net (`PLATFORM_FEE_RATE` in `lib/booking/offerPricing.ts`).

6. Insert booking; wash shops attach default `branch_id` if missing.

Slot collision is enforced by occupancy in `buildSlotsForShopDate` (client) plus merchant queue; stock of **time** is the slot, not a row lock.

3.3 End-to-end: retail catalog → cart → checkout

```
Home store tile
  → /store?category=spare_parts|accessories
    optional shopId → listStoreProductsByShop
  → StoreSearchHeader / StoreSubCategoryPills / StoreViewToggle
    products | location list | location map
  → StoreProductCard addProduct()
    guest → /auth-required
    signed-in → StoreCartContext (optimistic) → cart_items UPSERT
  → StoreCartCheckoutModal
    fulfillment: COD (address required, +50 EGP) | pickup
    stock-capped qty + overstock banner
    placeStoreOrder(..., p_shop_id) RPC
  → success alert; that shop's cart lines cleared server-side
  → order appears in owner StoreOrdersPanel via Realtime
  → customer tracks it on /(tabs)/orders (invoice / cancel pending / buy again)
```

Cart state: `context/StoreCartContext.tsx` backed by `lib/store/cartRepository.ts`. Unique key (`user_id`, `product_id`). Quantity is clamped to live stock in `lib/store/stockLimits.ts`.

3.4 Atomic store checkout (`place_store_order`)

Security definer RPC. Canonical body: `supabase/store_enhancements_production.sql` (also mirrored in `complete_system_schema.sql`). Client: `lib/store/productRepository.ts` `placeStoreOrder`. Signature includes optional `p_shop_id` so a mixed cart can check out one merchant at a time.

```
BEGIN (implicit in PL/pgSQL function)

1. AUTH
  auth.uid() IS NOT NULL
  else raise 'Not authenticated'

2. CART SHAPE
  optional p_shop_id filters cart lines to that merchant
  lock products FOR UPDATE (ordered by product id)
  cart not empty
  all products active and have shop_id
  DISTINCT shop_id count = 1
  that shop is_active

3. STOCK CHECK
  for each cart line:
    if products.stock_quantity < cart.quantity
      raise 'Insufficient stock for product %'

4. PRICE
  unit = coalesce(sale_price, price)
  subtotal = Σ unit * qty
  total = subtotal + delivery_fee

5. ORDER INSERT status='pending'
  store_orders (user_id, shop_id, customer_*, fulfillment, payment_method, totals)

6. LINES + STOCK RESERVE (decrement)
  insert store_order_items
  update products set stock_quantity = stock_quantity - qty
  where stock_quantity >= qty
  if not found → raise (concurrent checkout lost the race)
```

```

7. CLEAR CART
   delete cart_items where user_id = auth.uid()

8. RETURN order uuid

END

```

Reservation model: there is no separate `reservations` table. Stock is **decremented at order insert**. Cancellation **does restock** atomically: owner `update_store_order_status(..., cancelled)` and customer `cancel_own_store_order` lock product rows and increment `stock_quantity` by each line qty.

Single-merchant rule: mixed-shop carts fail with `Checkout supports products from one merchant at a time` unless the client passes `p_shop_id` and only that shop's lines are checked out.

3.5 Customer post-purchase

Artifact	Where
Service bookings	<code>/(tabs)/bookings</code> via <code>listBookingsForPhone</code>
Booking detail / invoice-style breakdown	<code>/booking/[id]</code> + <code>lib/booking/customerOrderPresentation.ts</code>
Store orders	<code>/(tabs)/orders</code> via <code>listStoreOrdersForUser</code> → <code>CustomerStoreOrdersList</code> / <code>CustomerStoreOrderCard</code>
Tax invoice (print / save)	<code>StoreOrderInvoiceModal</code> + <code>lib/store/storeOrderInvoice.ts</code> + <code>expo-print</code>
Customer cancel (pending only)	RPC <code>cancel_own_store_order</code> — restocks, then status <code>cancelled</code>
Buy again	<code>StoreCartContext.addProduct</code> from order lines
Reviews	<code>shop_reviews</code> after done booking (<code>addShopReview</code>)
Loyalty	<code>customer_merchant_loyalty</code> , <code>customer_wash_loyalty</code>
Notifications bell	<code>components/customer/CustomerNotificationsBell.tsx</code>

Customer booking progress UI: `components/customer/BookingProgressTimeline.tsx` .

4. Super admin governance panel

Route: `/admin` (`app/admin/index.tsx`)

Guard: `useShopAuth().isAdmin` else `admin_access_denied`

UI: `components/admin/AdminPanel.tsx`

Data: `lib/admin/adminRepository.ts`

Tabs (`AdminTab`): `dashboard` | `pending` | `merchants` | `moderation`

4.1 Merchant approval lifecycle

```

Merchant fills /welcome owner register
    |
    ▼
auth.signUp + RPC register_shop_owner
    |
    ├── inserts shops row   is_active = false
    ├── users.role         pending_owner
    └── users.shop_id      new shop
    |
    ▼

```

```

Login as merchant → ShopLoginResult 'pending_approval'
|
▼
Admin tab "pending" listPendingOwnerRequests()
  users.role = pending_owner JOIN shops
  |
  |— Accept (confirm dialog)
  |   AdminPanel.onApprove
  |   → functions.invoke('approve-merchant')
  |   → fallback RPC approve_shop_owner(p_target_user_id, p_target_shop_id)
  |   users.role = owner, is_active = true
  |   shops.is_active = true
  |
  |— Reject (confirm dialog)
  |   → functions.invoke('reject-merchant')
  |   → fallback RPC reject_shop_owner(...)
  |   pending shop/user torn down per RPC

```

Edge functions: `supabase/functions/approve-merchant/index.ts` , `supabase/functions/reject-merchant/index.ts` . They exist so Auth email confirmation / admin JWT can be applied server-side; the SQL RPCs are the data-plane fallback.

4.2 Active merchant management

List: `listActiveMerchants()` — `users.role=owner` , `is_active` , `shop is_active` , plus `shop_branches` count.

Control	Mechanism	Confirm?
Premium subscription toggle	RPC <code>admin_toggle_shop_premium(p_shop_id, p_is_premium)</code>	Yes (<code>admin_premium_confirm_*</code>)
Platform fee settlement	RPC <code>admin_settle_shop_platform_fees</code>	Ledger "settle" action
Branch oversight	<code>branchCount</code> on merchant row (read-only in list)	—
Account / shop removal	<code>deleteMerchant(shopId)</code>	Yes (<code>userConfirm</code>)
Feature flag (premium CMS)	<code>shops.is_premium</code>	Same toggle
Shop live flag	<code>shops.is_active</code> (approval sets true; delete removes shop)	—
Loyalty flag	<code>shops.is_loyalty_enabled</code>	Schema-level; not a dedicated admin toggle in <code>AdminPanel</code>

deleteMerchant sequence (client, admin RLS required):

1. Delete `shop_branches` for `shop_id`
2. Delete `products` for `shop_id`
3. Delete `store_orders` for `shop_id`
4. Delete `shops` row
5. Update `users` with that `shop_id` → `role='customer'` , `shop_id=null`

Bookings cascade from `shops` (`ON DELETE CASCADE ON bookings.shop_id`).

4.3 Dashboard & ledger

`fetchPlatformStats()` :

- Active shops count
- Completed bookings (`status=done`)

- Gross revenue from `final_amount_paid_egp`
- Reported community posts

`fetchMerchantLedger()` : per-shop completed bookings since `platform_fee_last_settled_at` , gross revenue, outstanding 12% fee.

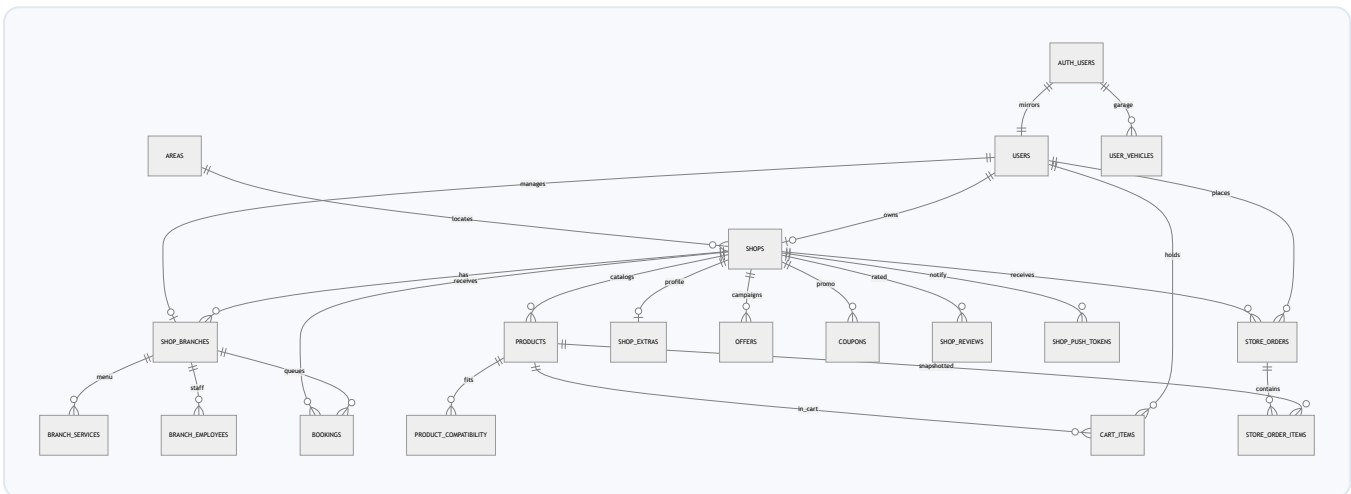
4.4 Moderation

Queue: reported `posts` , `comments` , `shop_reviews` . Actions: dismiss (`reported=false`) or delete row.

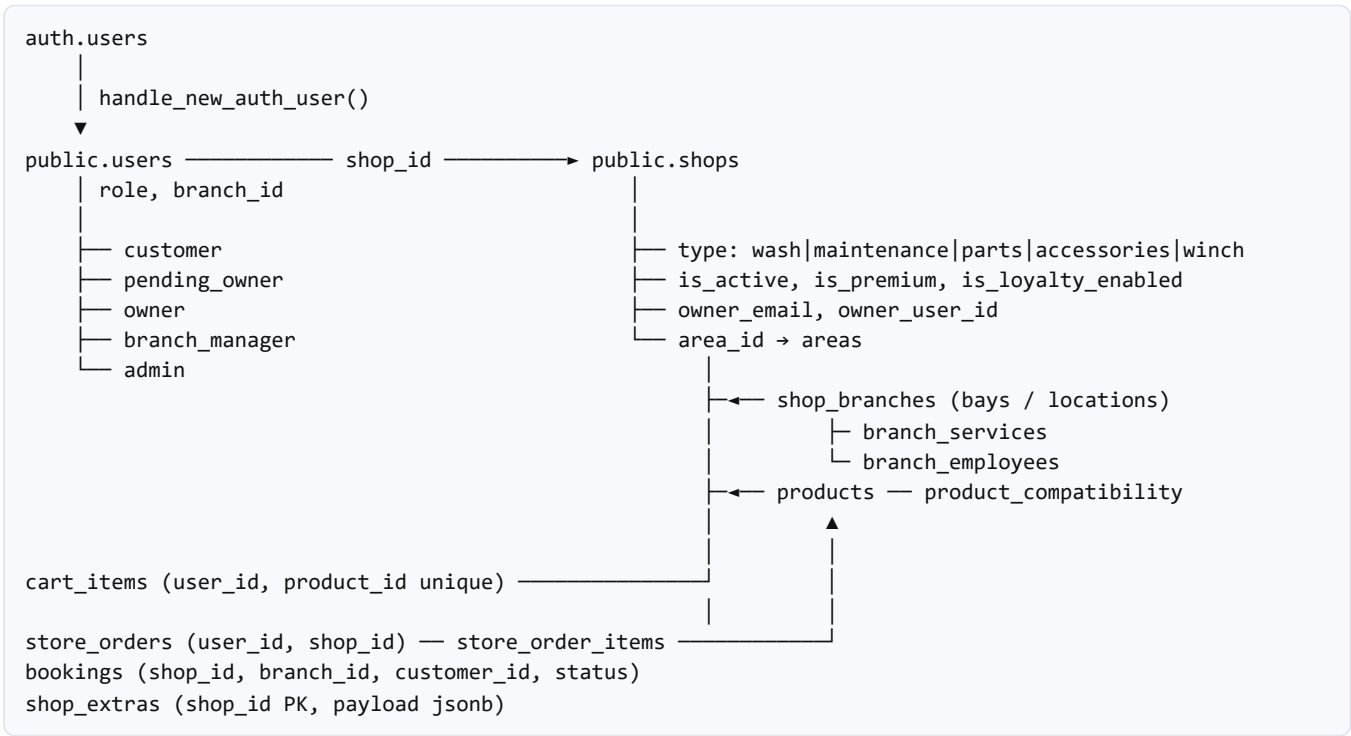
Secondary admin catalog: `/admin/store-products` .

5. Database schema & Supabase topology

5.1 Entity-relationship (Mermaid)



5.2 Entity-relationship (ASCII)



5.3 Enumerations

Type	Values
shop_type	maintenance , wash , parts , accessories , winch
user_role	customer , owner , branch_manager , admin , pending_owner
shop_operating_status	open , closed , busy , vacation
booking_status	pending , confirmed , in_progress , done , cancelled , no_show , suspended_by_shop
booking_type	app , walk_in
store_product_category	spare_parts , accessories
store_compatibility_type	universal , brand_specific , model_specific
store_fulfillment_method	cod , pickup , card
store_order_status	pending , preparing , ready , completed , cancelled

5.4 Table dictionaries

public.areas

Column	Type	Notes
id	text PK	Area slug
name / name_ar	text	Display
city / city_ar	text	City

public.shops

Column	Type	Notes
id	text PK	Tenant key used across app routes
name / name_ar	text	Brand
type	shop_type	Merchant category
category	text	Synced alias (car_wash , spare_parts , ...)
area_id	text FK → areas	Required in core schema
address / address_ar	text	
phone	text	
latitude / longitude	float	Map / proximity
owner_email	text	JWT fallback for is_shop_owner
owner_user_id	uuid FK → auth.users	Optional
rating	numeric	
is_active	boolean	Approval + operational flag
is_premium	boolean	SaaS / CMS gate
is_loyalty_enabled	boolean	Default true
platform_fee_last_settled_at	timestampz	Admin ledger
created_at / updated_at	timestampz	

Indexes: shops_owner_email_idx (lower email), shops_owner_user_id_idx , shops_category_active_idx .

public.users

Column	Type	Notes	
id	uuid PK FK → auth.users	Cascade delete	
email	text unique, lowercase check		
full_name / phone	text		
role	user_role	Default customer	
shop_id	text FK → shops	SET NULL on shop delete	
branch_id	uuid FK → shop_branches	Required if branch_manager	
preferred_locale	en \	ar	Step 18
is_active	boolean		
created_by	uuid	Staff invite provenance	
created_at / updated_at	timestamptz		

Constraints: owner must have shop_id ; branch_manager must have branch_id .

Indexes: users_role_idx , users_shop_id_idx , users_branch_id_idx , users_email_unique_idx .

public.shop_branches

Column	Type	Notes
id	uuid PK	
shop_id	text FK CASCADE	
slug	text	Unique per shop (main , ...)
name / name_ar	text	
area_id , address, phone, lat/lng	mixed	Override shop location
profile_* , more_info*	text	CMS
profile_image_url	text	Avatar
image_urls	jsonb	Gallery array
service_price_egp	numeric	Fallback price
service_duration_minutes	int default 60	Slot size
weekly_hours	jsonb	ShopDayHours[]
shop_status	shop_operating_status	Live bay status
vacation_mode	jsonb	
manager_user_id	uuid	Linked manager
is_active / is_default / sort_order	bool/int	
created_at / updated_at	timestamptz	

Indexes: shop_branches_shop_id_idx , shop_branches_manager_user_id_idx . Unique (shop_id, slug) .

public.branch_services

Menu items: name, name_ar, description*, category, price_egg, duration_minutes, visible, sort_order. FK shop_id, branch_id. Index branch_services_branch_id_idx.

public.branch_employees

No login. full_name, phone, job_title, notes, is_active. Index branch_employees_branch_id_idx.

public.products

Column	Type	Notes
id	uuid PK	
shop_id	text FK CASCADE NOT NULL	Merchant catalog
name / description	text	
category	store_product_category	
sub_category	text	Tag from constants
price	numeric(12,2) ≥ 0	Base price
sale_price	numeric	Optional, ≤ price
stock_quantity	int ≥ 0	Live stock
stock	generated	Alias of stock_quantity
image_url	text	Public Storage URL (cover; kept in sync with image_urls[1])
image_urls	text[] default {}, max 5	Gallery; trigger keeps cover + array aligned (store_enhancements_production.sql)
compatibility_type	enum	
rating / rating_count	numeric/int	
is_active	boolean	Hidden if false (owners still read via RLS)
created_at / updated_at	timestamptz	

Indexes: products_shop_active_idx (shop_id, is_active, created_at desc), products_low_stock_idx (shop_id, stock_quantity) WHERE is_active.

public.product_compatibility

product_id FK CASCADE, brand, model, year_start, year_end.

public.cart_items

Column	Type	Notes
id	uuid PK	
user_id	uuid FK auth.users CASCADE	
product_id	uuid FK products CASCADE	
quantity	int > 0	
unique	(user_id, product_id)	Upsert target

Indexes: cart_items_user_id_idx, unique cart_items_user_product_uidx.

public.store_orders

Column	Type	Notes
id	uuid PK	
user_id	uuid FK auth.users CASCADE	
customer_id	generated	Alias of user_id
shop_id	text FK shops RESTRICT NOT NULL	
subtotal / delivery_fee / total_price	numeric	
total_amount	generated	Alias of total_price
fulfillment_method	enum	cod / pickup / card
payment_method	text	CHECK in (cod , card , pickup)
status	store_order_status	Default pending
customer_name / customer_phone	text	Snapshot at checkout
delivery_address / delivery_notes / notes	text	
created_at / updated_at	timestampz	

Indexes: store_orders_shop_created_idx , store_orders_customer_created_idx , store_orders_shop_status_idx .

Replica identity: FULL (Realtime payloads).

public.store_order_items

order_id FK CASCADE, product_id SET NULL, product_name , unit_price , quantity , line_total , generated subtotal . Index store_order_items_order_idx .

public.bookings

Column	Type	Notes
id	uuid PK	
shop_id	text FK CASCADE	
branch_id	uuid FK SET NULL	Wash queue scope
shop_type	shop_type	Denormalized
customer_id	uuid SET NULL	
customer_phone / customer_name	text	
car_type / car_color	text	
service_id	uuid FK branch_services	
service_name / service_name_ar	text	Snapshot
service_price_egp / platform_fee_egp	numeric	
original_price_egp / points_redeemed / discount_applied_egp / final_amount_paid_egp	money/loyalty	
offer_id	uuid FK offers	
customer_notes / owner_rejection_note	text	
booking_type	app \	walk_in

Column	Type	Notes
scheduled_at	timestampz	Slot instant
booking_date	date	Generated Cairo date
time_slot	text	Generated HH24:MI
total_price	numeric	Synced from paid / service price
vehicle_details	jsonb	model/color/plate
status	booking_status	Default pending
created_at / updated_at	timestampz	

Indexes: bookings_shop_id_idx , bookings_branch_id_idx , bookings_customer_phone_idx , bookings_shop_scheduled_idx , bookings_customer_scheduled_idx , bookings_shop_status_idx , bookings_booking_type_idx , bookings_offer_id_idx .

Replica identity: FULL.

`public.shop_extras`

shop_id PK FK CASCADE, payload jsonb (profile, hours, gallery, services, store discount), updated_at . App mapper: lib/booking/shopExtrasStorage.ts / ShopExtras in lib/booking/types.ts .

Other commerce / social tables

Table	Purpose
offers	Shop campaigns (percent / dates / active)
coupons / coupon_usages	Promo codes + per-user log
shop_reviews	Ratings, owner reply, hidden, reported
posts / comments / post_votes / comment_likes	Driver network
customer_merchant_loyalty	Points per shop
customer_wash_loyalty	Stamp cards
user_vehicles / garage_snapshots	Customer garage
shop_push_tokens	Expo push targets
shop_report_history	Owner report HTML archive
customer_notifications / owner notification storage	Via lib/booking/commerceEvents.ts (table or local depending on migration)

5.5 RLS security model

RLS is enabled on shops, products, cart, orders, order items, bookings, extras, and related social tables.

Cart (cart_items): select/insert/update/delete only when `auth.uid() = user_id` .

Products:

- SELECT if `is_active` OR admin OR `is_shop_owner(shop_id)`
- INSERT/UPDATE/DELETE: admin or owning merchant (`unify-store-owner-data-access.sql` / `complete_system_schema.sql`)

Store orders:

- SELECT: buyer (`user_id`) OR admin OR shop owner

- UPDATE: admin or shop owner (status workflow via RPC, not a raw client UPDATE)
- INSERT: not granted to clients; **only** `place_store_order` (security definer)
- Customer cancel: **only** via `cancel_own_store_order` (pending + buyer + restock)

Order items: SELECT if parent order is readable by the same rules.

Bookings:

- INSERT: open (`Anyone` can create bookings) so guests-turned-customers and walk-in POS can write
- SELECT: `customer_id = auth.uid()` plus owner/manager policies from shop RLS (`can_manage_shop` / branch filters in later steps)
- Customer UPDATE: only to `cancelled`
- Owner UPDATE: status + rejection note via authenticated owner policies in `schema.sql` / step 2–4

Shop extras: public SELECT; mutating requires admin or owner.

Users: self-read plus `is_platform_admin()` .

Shops: public catalog read for active shops; admin read-all; owner manage own.

Policies are **defense in depth**. Stock mutation during checkout is not done with a client UPDATE; it is inside the RPC.

5.6 Storage buckets

`product-images` (`supabase/create-product-images-storage.sql`)

Property	Value
Public	yes
Size limit	10 MiB
MIME	jpeg, png, webp, gif, heic, heif
Object path	<code>shops/{shopId}/products/{file}</code>
SELECT	public
INSERT/UPDATE/DELETE	authenticated AND folder shop id matches owner (see <code>supabase/fix_storage_qls_product_images.sql</code>)

Uploader: `lib/supabase/storageUpload.ts` `uploadImageToBucket` . Inventory UI uses `throwOnError: true` .

Other media

Shop/branch galleries and covers also go through `uploadImageToBucket` with a configured bucket (`shop-images` in admin product form). Community media extensions: `apply-pitstop-2.0-step14d-community-media-extensions.sql` . Product rows store **URLs only**, never binary.

5.7 RPC functions (commerce & governance)

Function	Execute grant	Behavior
<code>place_store_order(fulfillment, fee, name, phone, address, notes, shop_id)</code>	authenticated	Atomic cart → order → stock → clear that shop’s cart lines. Optional <code>p_shop_id</code> .
<code>update_store_order_status(order_id, status)</code>	authenticated	Owner/admin. Restocks on cancel. Allows pending→ready and completed→cancelled (void).
<code>cancel_own_store_order(order_id)</code>	authenticated	Buyer only; pending → cancelled + restock
<code>get_store_owner_stats(shop_id)</code>	authenticated	JSON metrics; owner/admin. Revenue = completed only.

Function	Execute grant	Behavior
<code>get_store_sales_report(shop_id, from, to)</code>	authenticated	Completed-order sales report for <code>/shop/store-reports</code>
<code>register_shop_owner(...)</code>	authenticated	Inactive shop + <code>pending_owner</code>
<code>approve_shop_owner(user_id, shop_id)</code>	authenticated (admin inside)	Activate merchant
<code>reject_shop_owner(user_id, shop_id)</code>	authenticated (admin inside)	Reject request
<code>admin_toggle_shop_premium</code>	admin	Feature flag
<code>admin_settle_shop_platform_fees</code>	admin	Ledger timestamp
<code>is_shop_owner</code> / <code>is_platform_admin</code> / <code>can_manage_shop</code> / <code>can_manage_branch</code>	internal	RLS helpers
Branch-manager RPCs	step 3 SQL	Invite / link / remove manager

`get_store_owner_stats` JSON keys: `total_revenue` , `total_orders` , `pending_orders` , `total_products` , `low_stock_count` . Client maps `snake_case` and `camelCase` (`storeStatsRepository.ts`).

6. Realtime event pipelines & edge integrations

6.1 Publication

`complete_system_schema.sql` adds to `supabase_realtime` :

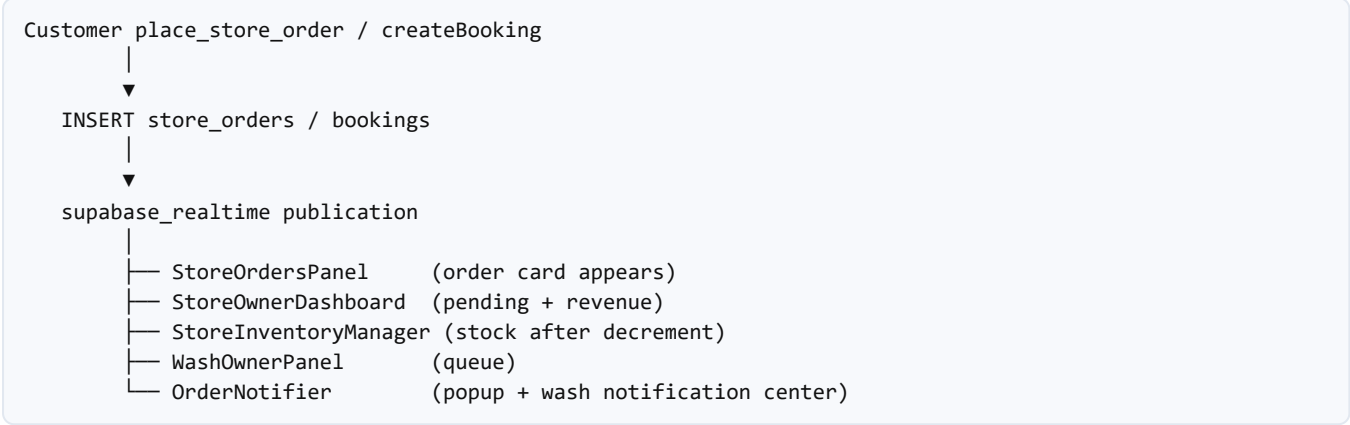
- `public.store_orders` (replica identity FULL)
- `public.bookings` (FULL)
- `public.products` (FULL)

Additional app listeners also subscribe to `shops` and `shop_branches` (`lib/booking/shopMapDiscovery.ts`).

6.2 Channel catalog

Channel name	Table / event	Consumer	Effect
<code>store-dashboard:{shopId}</code>	<code>store_orders</code> *, <code>products</code> *	<code>StoreOwnerDashboard</code>	Refresh metrics
<code>store-inventory:{shopId}</code>	<code>products</code> *	<code>StoreInventoryManager</code>	Reload SKUs / stock
<code>store-orders:{shopId}</code>	<code>store_orders</code> *	<code>StoreOrdersPanel</code>	New/updated orders
<code>owner-bookings:{shopId}</code>	<code>bookings</code> *	<code>WashOwnerPanel</code>	Silent workspace refresh
<code>merchant-bookings:{shopId}</code> (via <code>subscribeMerchantBookingRealtime</code>)	<code>bookings</code> INSERT/UPDATE	<code>OrderNotifier</code> , wash hub	In-app alert, vibration, pending queue
Map discovery channel	<code>shops</code> , <code>shop_branches</code> *	<code>shopMapDiscovery.ts</code>	Live map pins
Offers channel	<code>offers</code>	<code>offerRepository.ts</code>	Live campaign badges

Filters are typically `shop_id=eq.{id}` so tenants do not receive other merchants' rows. RLS still applies to Realtime payloads.



6.3 In-app merchant notifier

- `components/merchant/OrderNotifier.tsx` (headless) + `lib/notifications/notificationService.ts` :
- Scopes rows for branch managers
 - Distinguishes pending insert vs cancellation
 - `userAlert` + optional `Vibration`
 - Wash center rows via `pushWashCenterNotification` (`lib/booking/wash/washNotificationCenter.ts`)
 - Hub: `/shop/wash-owner-hub?tab=orders`

6.4 Push notifications

Registration: `lib/push/shopPush.ts` `registerOwnerPushToken` on shop tab focus (`app/(tabs)/shop.tsx`).

Table: `shop_push_tokens` (`shop_id` , `owner_email` , `expo_push_token` , `locale` , optional `branch_id`). Unique (`shop_id` , `expo_push_token`) .

Delivery: HTTP POST `https://exp.host/--api/v2/push/send` from the app/server helper (not an Edge Function). Copy is bilingual (new booking / new parts order).

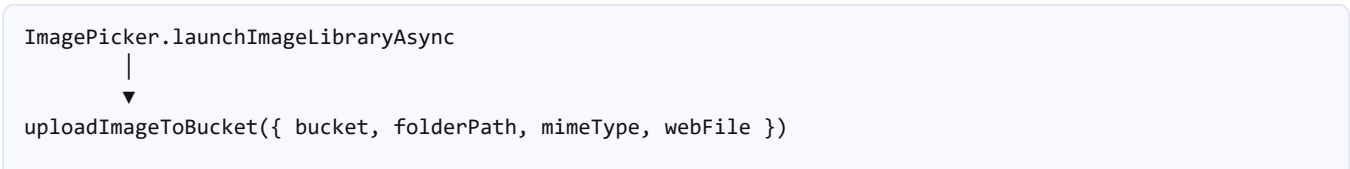
Customer notifications: `pushCustomerNotification` in `lib/booking/commerceEvents.ts` (approved, declined, reminder, shop reopened, parts order, review reply). Reminders: `lib/booking/bookingReminders.ts` (local scheduling).

6.5 Edge Functions

Function	Path	Role
<code>approve-merchant</code>	<code>supabase/functions/approve-merchant/index.ts</code>	Admin JWT → confirm Auth user + RPC activate shop
<code>reject-merchant</code>	<code>supabase/functions/reject-merchant/index.ts</code>	Admin JWT → reject pending merchant
<code>send-booking-confirmation</code>	<code>supabase/functions/send-booking-confirmation/index.ts</code>	Email after <code>handleBookingConfirmed</code>

Client invoke: `lib/admin/adminRepository.ts` (approve/reject with session `Authorization` header), `lib/booking/sendBookingConfirmationEmail.ts` .

6.6 File storage workflow



```

| fetch local URI → Blob / ArrayBuffer
| storage.from(bucket).upload(path, body, { upsert: false })
| getPublicUrl(path)
|
|
|

```

persist URL(s) on products.image_url + products.image_urls OR shop_extras / branch image_urls

Path convention for products: `shops/{shopId}/products/{epoch}-{stem}-{rand}.{ext}`. Owners cannot write another shop's folder (Storage RLS on folder segment 2 = `shops.id`).

6.7 Email & other integrations

Integration	Trigger
Booking confirmation email	Status → <code>confirmed</code> (non walk-in)
Expo Push	New booking / parts order to registered owner tokens
OpenAI (optional)	Historical assistant; current tab is driver network
OSM / Leaflet	<code>OsmLocationPicker</code> , <code>CategoryShopsMap</code> for discovery and branch pins
PostGIS	Nearby shop queries (step 15a)

Appendix A — Status transition cheat sheets

Store order (`store_order_status`)

From	To	Actor	RPC / UI
(none)	<code>pending</code>	Customer	<code>place_store_order</code>
<code>pending</code>	<code>preparing</code>	Owner	<code>update_store_order_status</code>
<code>pending</code>	<code>ready</code>	Owner	same (skip processing)
<code>pending</code>	<code>cancelled</code>	Owner or customer	Owner RPC, or <code>cancel_own_store_order</code> (pending only). Restocks.
<code>preparing</code>	<code>ready</code>	Owner	same
<code>preparing</code>	<code>cancelled</code>	Owner	same; restocks
<code>ready</code>	<code>completed</code>	Owner	same; now counts as revenue
<code>ready</code>	<code>cancelled</code>	Owner	same; restocks
<code>completed</code>	<code>cancelled</code>	Owner	void + restock; drops out of revenue

Booking (`booking_status`)

From	To	Actor	Notes
(none)	<code>pending</code>	Customer / walk-in POS	<code>createBooking</code>
<code>pending</code>	<code>confirmed</code>	Owner	Email + customer notification
<code>pending</code>	<code>cancelled</code>	Owner or customer	Rejection note optional
<code>confirmed</code>	<code>in_progress</code>	Owner	Active bay work
<code>confirmed</code> / <code>in_progress</code>	<code>done</code>	Owner or virtual 60 min	Loyalty earn on persisted done
<code>confirmed</code>	<code>no_show</code>	Owner	Override prompt

From	To	Actor	Notes
any operational	suspended_by_shop	Shop closure	Step 17

Merchant account

From	To	Actor
(none)	pending_owner + inactive shop	Self-register
pending_owner	owner + active shop	Admin accept
pending_owner	removed	Admin reject
owner	customer (shop deleted)	Admin deleteMerchant
owner	branch_manager accounts	Owner invite (premium wash)

Appendix B — Key source map

Concern	Files
Root navigation	app/_layout.tsx , app/(tabs)/_layout.tsx
Auth	context/CustomerAuthContext.tsx , context/ShopAuthContext.tsx , lib/shop/shopStaffUser.ts
Store owner UI	StoreOwnerDashboard.tsx , StoreInventoryManager.tsx , StoreOrdersPanel.tsx , StoreOwnerSettings.tsx , StoreOwnerProfileSections.tsx , AddProductModal.tsx
Customer store orders	app/(tabs)/orders.tsx , CustomerStoreOrdersList.tsx , CustomerStoreOrderCard.tsx , StoreOrderInvoiceModal.tsx
Wash owner UI	components/owner/wash/WashOwnerPanel.tsx , app/shop/wash-owner-hub.tsx
Service owner UI	app/(tabs)/shop.tsx
Dashboard tab config	lib/owner/dashboardConfig.ts
Cart / checkout	context/StoreCartContext.tsx , StoreCartCheckoutModal.tsx , lib/store/stockLimits.ts , StoreQuantityStepper.tsx
Catalog	app/store.tsx , app/parts-shop/[shopId].tsx , StoreProductCard.tsx , StoreProductImageSlider.tsx , lib/store/productImages.ts
Sales reports	app/shop/store-reports.tsx , lib/store/salesReportRepository.ts
Booking checkout	app/book/[shopId].tsx , lib/booking/storage.ts
Admin	components/admin/AdminPanel.tsx , lib/admin/adminRepository.ts
Schema	supabase/schema.sql , supabase/complete_system_schema.sql
Stats / lifecycle SQL	supabase/store_order_lifecycle_refactor.sql , supabase/store_enhancements_production.sql , supabase/customer_cancel_store_order.sql , supabase/fix-store-owner-stats-rpc.sql
Storage	supabase/create-product-images-storage.sql , supabase/fix_storage_rls_product_images.sql , lib/supabase/storageUpload.ts
Types	lib/supabase/database.types.ts , lib/store/types.ts , lib/booking/types.ts

Appendix C — Operational notes for implementers

1. **Do not insert `store_orders` from the client.** Always `rpc('place_store_order')`.
2. **Do not decrement or restock stock in the app.** Checkout and cancel RPCs row-lock products (`FOR UPDATE`).
3. **Cart checkout is single-shop.** Pass `p_shop_id` to check out one merchant from a mixed cart.
4. **SKU media is multi-image.** Cover `image_url` plus `image_urls` (max 5). Shop/branch galleries remain separate JSON arrays.
5. **Revenue is completed-only.** Pending / preparing / ready are operational, not GMV.
6. **Cancel restocks.** Owner cancel at any non-cancelled status, and customer cancel while pending, both increment stock.
7. **Customer store orders live on `/(tabs)/orders`** , not on the bookings tab.
8. **Delivery zones are not modeled.** COD uses a customer-typed address and a flat 50 EGP fee. Pickup delivery line is Free / مجاناً on the tax invoice.
9. **“Bays” = `shop_branches` + `slot occupancy`**, not a separate bay entity.
10. **Premium is the main merchant feature flag** (`shops.is_premium`), toggled only by admins.
11. **Realtime requires replica identity FULL** on `store_orders` , `bookings` , and `products` for complete row payloads.
12. **Virtual `done` is display-only** until an owner persists status; do not book revenue on `lifecycleAutoCompleted` .
13. **Admin delete is destructive** and confirmation-gated in UI; bookings cascade with the shop.

End of document.